TP 3

JavaScript et jQuery : Dessins, animations et formulaires

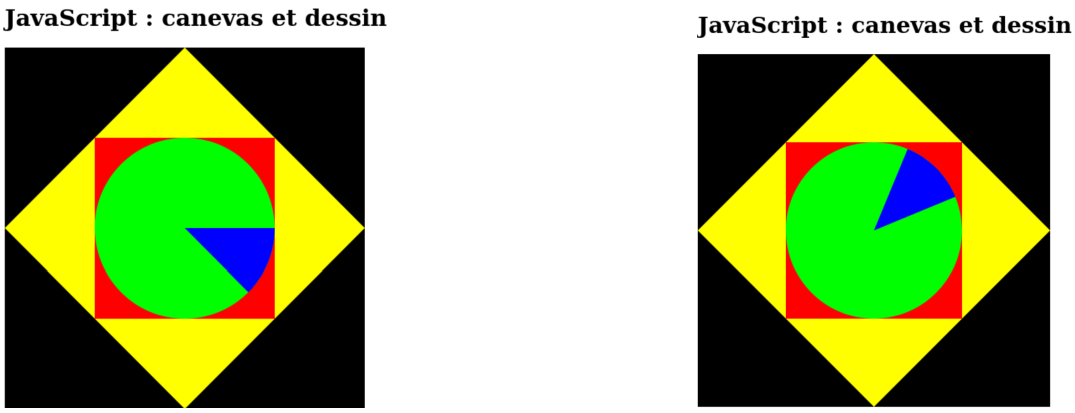
Dans ce TP, nous allons continuer la programmation Javascript, puis aborder la programmation jQuery.

——————————————— * JavaScript * ———————————————

Cette partie est dédiée à JavaScript. Sauf mention contraire, aucune modification des fichiers HTML fournis n'est autorisée.

EXERCICE

Exercice 3.1 (Dessin animé)**.** L'objectif est de réaliser (en JavaScript) un dessin similaire à celui de la figure 3.1a.



(a) Dessin initial

(b) Dessin après 3 clics

FIGURE 3.1 – Dessin

Question 1 Télécharger le fichier `dessin.html`. Créer un fichier `dessin.js`. On ne travaillera que dans ce fichier JavaScript. Implémenter une méthode `dessine()` qui dessine la forme de la figure 3.1a grâce à JavaScript. Quelques fonctions utiles : `fillStyle`, `fillRect`, `beginPath`, `moveTo`, `closePath`, `lineTo`, `arc`... (Voir le cours.) Le carré noir fait 500 px de côté (on pourra utiliser une constante). Comme d'habitude, appeler cette méthode dans un auditeur d'évènement sur l'évènement `'load'`, par exemple :

```
window.addEventListener("load", () => {
  dessine();
}
```

Question 2 Ajouter des gestionnaires d'évènement pour que :

— un clic de souris décale l'arc de cercle bleu de $\frac{\pi}{8}$ dans le sens trigonométrique (antihoraire) ;

— la pression de la touche « l » décale l'arc de cercle bleu de $\frac{\pi}{8}$ dans le sens trigonométrique ;

— la pression de la touche « d » décale l'arc de cercle bleu de $\frac{\pi}{8}$ dans le sens antitrigonométrique (sens horaire).

Indications : ajouter une variable globale `theta` qui encode l'angle initial de l'arc. Puis implémenter deux méthodes `decaleAntiTrigo()` et `decaleTrigo()` qui changent la valeur de `theta`, puis rappellent `dessine`. Enfin, ajouter dans les auditeurs concernés les appels à ces deux fonctions, par exemple :

```
document.addEventListener('keypress', (e) => {if(e.key === 'g'){
    decaleTrigo(); }});
```

Un exemple de rendu après 3 clics de souris est donné sur la figure 3.1b.

Question (optionnelle) 3 Ajouter un contrôle par les flèches du clavier, en plus des touches « l » et « g ».

Question (optionnelle) 4 Ajouter un auditeur responsable du fait que, toutes les 2 secondes, l'arc de cercle bleu se décale dans le sens trigonométrique.

Question (optionnelle) 5 Ajouter deux boutons radio « Sens horaire » et « Sens antihoraire » dans la page HTML (qu'il est autorisé de modifier). (Aucun bouton « envoyer » n'est demandé, ni même un élément `<form>`.)

En fonction de la sélection du bouton, les clics de souris suivants doivent occasionner une rotation horaire ou antihoraire.

■

EXERCICE

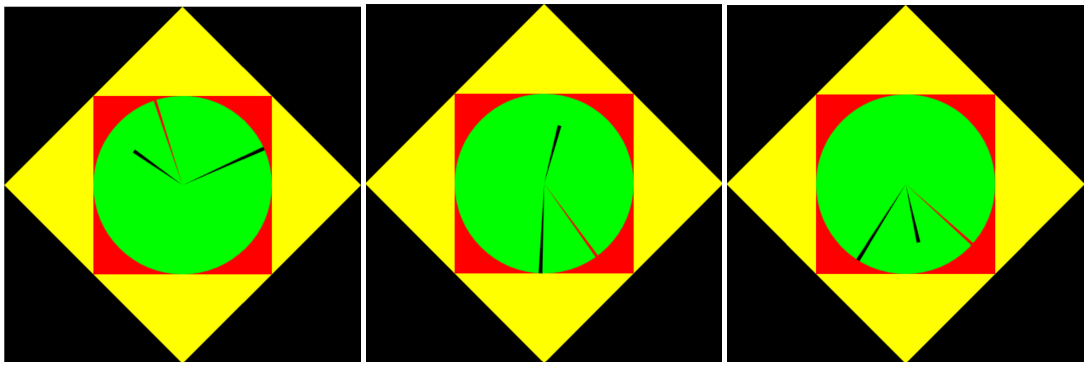
Exercice 3.2 (**(Optionnel)** - Horloge)**.** L'objectif est de réaliser (en JavaScript) une horloge similaire à celle de la figure 3.2, à partir de l'exercice précédent.

Question 1 Dupliquer les fichiers `dessin.html` et `dessin.js` (afin de ne pas supprimer votre exercice précédent) et les nommer `horloge.html` et `horloge.js` (ne pas oublier d'appeler depuis `horloge.html` votre bon fichier JavaScript, donc `horloge.js`).

Supprimer l'arc bleu (ainsi que toute méthode ou variable associée à cet arc) ; celui-ci n'est plus utile dans cet exercice.

Question 2 Ajouter trois variables globales encodant les angles courants des trois aiguilles de l'horloge :

```
let theta_aiguille_secondes;
let theta_aiguille_minutes;
let theta_aiguille_heures;
```



(a) À 10h10 (b) À 12h30 (c) À 17h35

FIGURE 3.2 – Horloge

Question 3 Dessiner les trois aiguilles sous forme de trois arcs de cercle (en suivant la technique de l'arc bleu à l'exercice précédent). Varier les couleurs, longueurs et épaisseurs.

Sur la figure 3.2, l'aiguille des secondes est très fine (largeur d'arc : $\frac{\pi}{100}$) et rouge, l'aiguille des minutes est fine ($\frac{\pi}{75}$) et noire, et l'aiguille des heures est plus épaisse ($\frac{\pi}{50}$) et un peu plus courte ($\frac{2}{3}$ de la longueur des autres aiguilles).

Pour tester le rendu, affecter les valeurs des 3 variables encodant les angles initiaux (définies à la question précédente) à des valeurs différentes arbitraires.

Question 4 Créer une méthode `actualiseHeure()` qui actualise les angles des trois aiguilles en fonction de l'heure exacte. Regarder la documentation de `Date`. Programmer ensuite l'exécution de cette méthode toutes les secondes. Ne pas oublier de redessiner toute l'horloge à la fin de l'exécution de `actualiseHeure()`.

Note : pour programmer un évènement récurrent, on utilise la construction suivante :

```
// Variable globale
let refreshIntervalIdAiguille;
// ...
window.addEventListener('load', function() {
  // ...
  clearInterval(refreshIntervalIdAiguille);
  refreshIntervalIdAiguille = window.setInterval(methode_a_appeler,
    periode_en_millisecondes);
  // ...
}
```

Question 5 Normalement, pendant la première seconde de l'affichage de la page Web, l'horloge sera à la mauvaise heure. Que faut-il faire (en une seule ligne de code) pour que, dès l'affichage de la page Web, l'horloge s'affiche à la bonne heure ?

Question 6 Dans la méthode `actualiseHeure()`, ajuster précisément les angles des aiguilles des heures et des minutes afin que ceux-ci dépendent aussi des minutes et des secondes (c'est-à-dire que l'aiguille des heures ne bouge pas d'un cran toutes les heures, mais s'ajuste précisément chaque seconde).

Par exemple, noter la bonne position de l'aiguille des heures (à mi-chemin entre 12h et 13h) sur l'horloge affichant 12h30 (figure 3.2b).

C Ajouter des animations personnalisées, par exemple une aiguille des secondes qui avance de façon continue (et non par secondes), ou bien qui fonctionne par à-coups (c'est-

à-dire que, à chaque seconde, l'aiguille avance, mais avec une légère vibration, de façon analogue à une horloge physique).

■

* jQuery *

Cette partie est dédiée à jQuery. Sauf mention contraire, l'utilisation de jQuery est obligatoire pour toutes les manipulations DOM et les évènements.
Le code HTML et CSS doit être propre et valide.
Pensez à utiliser la console du navigateur (F12) pour le débogage.

EXERCICE

Exercice 3.3 (Mise en place et Sélecteurs)**.** L'objectif est de configurer l'environnement jQuery et de tester les sélecteurs de base.

Question 1 Créer une page HTML minimale `jquery_init.html`. Inclure la bibliothèque jQuery via le CDN (Content Delivery Network) comme recommandé dans le cours. Ajouter un fichier script externe `init.js`.

Question 2 Dans `init.js`, utiliser la fonction `$(document).ready(...)` (ou sa version courte) pour s'assurer que le DOM est chargé avant d'exécuter le code. Ajouter un `console.log("jQuery est prêt !")`; pour vérifier le bon fonctionnement.

Question 3 Dans le HTML, ajouter :
- Un titre `<h1>` avec le texte "Découverte de jQuery".
- Trois paragraphes `<p>`.
- Une liste `<ul>` avec 4 éléments `<li>`.
- Un bouton `<button id="btn-style">Changer le style</button>`.

Question 4 Dans le fichier JS, ajouter un écouteur d'évènement click sur le bouton. Lorsque l'on clique dessus :
- Le titre h1 change de couleur (utiliser `.css()`).
- Les paragraphes prennent une bordure de 1px solid black.
- Les éléments de liste `<li>` en position impaire (odd) prennent un fond gris clair (utiliser le sélecteur `:odd` ou `:even`).

■

EXERCICE

Exercice 3.4 (Le bouton capricieux)**.** L'objectif est de créer une interface ludique où l'utilisateur est "forcé" de cliquer sur une réponse spécifique.

Question 1 Créer une page HTML contenant :
- Une question dans une balise `<h2>` (par exemple : "Êtes-vous satisfait de votre salaire?" ou "Aimez-vous jQuery?" ou "Veux-tu m'épouser?" ...).
- Un bouton `<button id="btn-oui">OUI</button>`.
- Un bouton `<button id="btn-non">NON</button>`.

— **Important :** En CSS, assurez-vous que le bouton "NON" possède une `position: absolute;` pour pouvoir être déplacé n'importe où sur la page via le script.

Question 2 Écrire le script jQuery pour le bouton "NON" :
— Utiliser l'évènement `mouseenter` ou `mouseover`
— Dès que la souris survole le bouton, celui-ci doit se déplacer instantanément à une position aléatoire dans la fenêtre.
— *Indice :* Utiliser `Math.random()` couplé à `$(window).width()` et `$(window).height()` pour calculer de nouvelles coordonnées `top` et `left`. Appliquer ces valeurs avec la méthode `.css()`.

Question 3 Écrire le script jQuery pour le bouton "OUI" :
— Au survol de la souris `mouseenter`, le bouton doit grossir légèrement (taille et police)
— Lorsque la souris quitte le bouton `mouseleave` il doit reprendre sa taille initiale.
— *Indice :* Vous pouvez utiliser `.css()` ou `.animate()` pour une transition plus fluide.

Question 4 Gérer le clic sur le bouton "OUI" :
— Lorsque l'utilisateur clique enfin sur "OUI", tout le contenu de la page (ou du conteneur principal) doit être remplacé par un message de succès et une image (GIF animé ou vidéo) festive.
— *Indice* Utiliser `.html()` pour remplacer le contenu, ou `.empty()` suivi de `.append()`.

Question (optionnelle) 5 Faire en sorte que si l'utilisateur essaie de "tricher" en utilisant la touche tabulation pour atteindre le bouton "NON", celui-ci se déplace également lorsqu'il reçoit le focus (`focus`).

■

EXERCICE

Exercice 3.5 (Le "To-Do List" dynamique)**.** Cet exercice porte sur la manipulation du DOM (ajout/suppression).

Question 1 Créer une page `todo.html` contenant :
— Un champ de texte (`<input type="text" id="tache">`)
— Un bouton "Ajouter" (`<button id="add">`)
— Une liste vide (`<ul id="liste-taches">`)

Question 2 Écrire le script jQuery qui effectue l'action suivante au clic sur le bouton "Ajouter" :
— Récupérer le texte saisi dans l'input (utiliser `.val()`).
— Si le texte n'est pas vide, créer un nouvel élément `<li>` contenant ce texte et l'ajouter à la fin de la liste `liste-taches`.
— Vider le champ de texte après l'ajout.

Question 3 Faire en sorte que si l'on clique sur un élément `<li>` de la liste, celui-ci disparaisse (soit supprimé du DOM).
Attention : Comme les éléments `<li>` sont créés dynamiquement après le chargement de la page, un simple `$("li").click(...)` ne fonctionnera pas pour les nouveaux éléments. Il faut utiliser la délégation d'évènements avec `.on()` sur le parent.

Question (optionnelle) 4 Ajouter deux boutons globaux :
- "Tout vider" : supprime tous les éléments de la liste (`.empty()`).
- "Masquer/Afficher" : masque ou affiche a liste complète avec un effet de fondu (`.fadeToggle()`).

■

EXERCICE

Exercice 3.6 (FAQ Animée - Accordéon)**.** Cet exercice exploite les effets d'animation avec jQuery.

Question 1 Créer une structure HTML pour une Foire Aux Questions (FAQ)

```
<div class="faq-item">
<h3 class="question">Quelle est la capitale de la France ?</h3>
<p class="reponse">Paris est la capitale de la France.</p>
</div>
<div class="faq-item">
    <h3 class="question">Qu'est-ce que jQuery ?</h3>
    <p class="reponse">Une bibliothèque JavaScript rapide et
concise.</p>
</div>

```

Question 2 En CSS (fichier externe ou balise style), masquer par défaut toutes les réponses `display: none;`). Styliser les questions pour qu'elles ressemblent à des boutons ou des zones cliquables.

Question 3 En jQuery, faire en sorte que :
- Au clic sur une question, la réponse associée s'affiche en glissant vers le bas (`slideDown` ou `slideToggle`)
- *Indice :* Pensez a l'élément suivant de la question `.next()`.

Question (optionnelle) 4 Améliorer le script pour créer un véritable effet *accordéon* : si on ouvre une question, toutes les autres réponses doivent se fermer automatiquement.
Indices :
- `slideUp`
- `.not(this)` ou `.siblings()`.

■

EXERCICE

Exercice 3.7 (Manipulation de classes et CSS)**.**

Question 1 Créer un formulaire simple avec 3 champs (Nom, Prénom, Email) et un bouton "Envoyer".

Question 2 En CSS, créer une classe `.erreur` qui met une bordure rouge et un fond rouge clair au champ input.

Question 3 Au clic sur "Envoyer", parcourir tous les champs input.

- Si un champ est vide, lui ajouter la classe `.erreur` (utiliser `.addClass()`).
- Si un champ est rempli, lui retirer la classe `.erreur` (utiliser `.removeClass()`)
- *Indice :* Utiliser `$(".mon-formulaire input").each(function()...)` pour parcourir les éléments

Question (optionnelle) 4 Faire trembler le formulaire si une erreur est détecter (utiliser `.animate()` pour déplacer légèrement le formulaire à gauche puis à droite rapidement.

■